

2026 SISBID Dimension Reduction Lab

Genevera I. Allen & Yufeng Liu

2026-07-24

PCA / Dimension Reduction Lab Using Digits Data

Lab goals

In this lab, you will compare several dimension reduction methods on handwritten digit images. The main goals are to:

- explore how different methods represent high-dimensional image data in two or more dimensions;
- interpret components, loadings, basis images, and embeddings;
- compare methods in terms of visualization, interpretability, sensitivity to tuning parameters, and usefulness for this data set;
- practice choosing tuning parameters and justifying those choices.

Data set

The primary data set is the digits data in `UnsupL_SISBID_2026.Rdata`. Each row corresponds to a digit image and each column corresponds to a pixel. The helper function `imagedigit()` is included in the data file and can be used to display an image from a length-256 vector.

You may use all digits, but many algorithms will run faster if you begin with a subset. Looking at 3's, 5's, and 8's is often interesting because these digits can have visually similar shapes.

Getting Started

The starter code below is intentionally not exhaustive. It is meant to help you begin, not to fully complete the lab.

For each problem, consider giving:

1. the code needed to run your analysis;
2. one or more plots or visual summaries;
3. a short interpretation of what the method reveals;
4. a brief discussion of tuning choices, when relevant.

Problems

Problem 1: PCA

Problem 1a. Apply PCA to the digits data. Be explicit about whether you center and/or scale the data, and explain your choice.

Problem 1b. Do the first several PCs separate different digits well? Why or why not?

Problem 1c. Use the first several PCs and PC loadings to evaluate the major patterns in the digits data. Can you describe the image pattern represented by each of the first five PCs?

Problem 1d. How many PCs are needed to explain 95% of the variance? Which number of PCs would you retain for interpretation or visualization, and why?

Optional extension. Use a small number of PCs to reconstruct several digit images. How does the reconstruction improve as you increase the number of PCs?

Problem 2: MDS

Problem 2a. Apply classical MDS to this data. Try several distance metrics and different numbers of MDS components.

Problem 2b. Which distance metric works best for this data? Which one reveals the most separation among digits?

Problem 2c. Compare the MDS maps to the PCA maps. Which is preferable for this data set, and why?

Problem 3: NMF

Problem 3a. Apply NMF to this data set.

Problem 3b. Which value of K did you use? Why? What happens when you slightly change your chosen value of K ?

Problem 3c. Interpret the NMF basis factors. Are they easier or harder to interpret than PCA or ICA components?

Problem 4: UMAP

Problem 4a. Apply UMAP to this data set.

Problem 4b. Try different distance metrics and different numbers of neighbors. How do these tuning choices change the result?

Problem 4c. Interpret the UMAP embedding. Does it identify meaningful digit groups?

Problem 5: t-SNE

Problem 5a. Apply t-SNE to this data set.

Problem 5b. Try changing the perplexity. How does this change your result?

Problem 5c. Interpret the t-SNE embedding. Does it identify meaningful digit groups?

Problem 6: Comparisons

Problem 6a. Compare PCA, MDS, NMF, UMAP, and t-SNE on this data set. Which method best separates the different digits? Which method reveals the most interpretable patterns?

Problem 6b. Overall, which method do you recommend for this data set and why?

Starter R code

Setup

Do not peek at this section if you want to practice coding entirely on your own. The code below is meant to get you started, but you should modify it, add to it, and explain your choices.

```
rm(list = ls())

knitr::opts_chunk$set(
  echo = TRUE,
  message = FALSE,
  warning = FALSE,
  fig.width = 7,
  fig.height = 5
)

library(ggplot2)

# Optional packages used later in the lab. Install any missing packages as needed:
# install.packages(c("fastICA", "NMF", "umap", "Rtsne", "GGally"))
optional_packages <- c("fastICA", "NMF", "umap", "Rtsne", "GGally")
missing_packages <- optional_packages[!vapply(optional_packages, requireNamespace, logical(1), quietly = TRUE)]
if (length(missing_packages) > 0) {
  message("Optional packages not installed: ", paste(missing_packages, collapse = ", "))
}
```

Load and inspect the data

```
load("UnsupL_SISBID_2026.Rdata")

# Check what objects were loaded.
ls()

## [1] "author"          "cdat"             "digits"
## [4] "gdmat"           "imagedigit"       "missing_packages"
## [7] "optional_packages" "sachscov"         "sachsdmat"

dim(digits)

## [1] 9297 256

table(rownames(digits))

##
##   0    1    2    3    4    5    6    7    8    9
## 1553 1269  929  824  852  716  833  792  708  821
```

Helper functions

```
plot_digit_grid <- function(X, labels = NULL, n = 32, main = "Digit examples") {
  n <- min(n, nrow(X))
  old_par <- par(no.readonly = TRUE)
  on.exit(par(old_par))
  par(mfrow = c(ceiling(n / 8), 8), mar = c(0.1, 0.1, 1.2, 0.1))
  for (i in seq_len(n)) {
    imagedigit(X[i, ])
    if (!is.null(labels)) title(main = labels[i], line = -1)
  }
  mtext(main, outer = TRUE, line = -1)
}
```

```

plot_embedding <- function(embedding, labels, title = "Embedding", xlab = "Dimension 1", ylab = "Dimension 2") {
  plot_df <- data.frame(
    dim1 = embedding[, 1],
    dim2 = embedding[, 2],
    digit = factor(labels)
  )

  ggplot(plot_df, aes(x = dim1, y = dim2, color = digit, label = digit)) +
    geom_point(alpha = 0.45, size = 1.6) +
    geom_text(check_overlap = TRUE, size = 3, show.legend = FALSE) +
    labs(title = title, x = xlab, y = ylab, color = "Digit") +
    coord_equal() +
    theme_minimal()
}

plot_basis_images <- function(B, n_show = 10, main = "Basis / loading images") {
  n_show <- min(n_show, ncol(B))
  old_par <- par(no.readonly = TRUE)
  on.exit(par(old_par))
  par(mfrow = c(ceiling(n_show / 5), 5), mar = c(0.1, 0.1, 1.2, 0.1))
  for (i in seq_len(n_show)) {
    imagedigit(B[, i])
    title(main = paste0("Component ", i), line = -1)
  }
  mtext(main, outer = TRUE, line = -1)
}

```

Choose a working subset

Start with a small, interpretable subset. After you understand the code, try changing `digits_to_use` or using all digits.

```

set.seed(1)

digit_labels_all <- rownames(digits)
digits_to_use <- c("3", "5", "8") # TODO: try other choices, or use all unique labels.
max_per_digit <- 200               # TODO: increase or set to Inf if your computer is fast.

idx <- unlist(lapply(digits_to_use, function(d) {
  candidates <- which(digit_labels_all == d)
  sample(candidates, size = min(length(candidates), max_per_digit))
}))

idx <- sort(idx)
dat_lab <- digits[idx, , drop = FALSE]
digit_lab <- factor(digit_labels_all[idx])

cat("Working data dimensions:", dim(dat_lab), "\n")

## Working data dimensions: 600 256

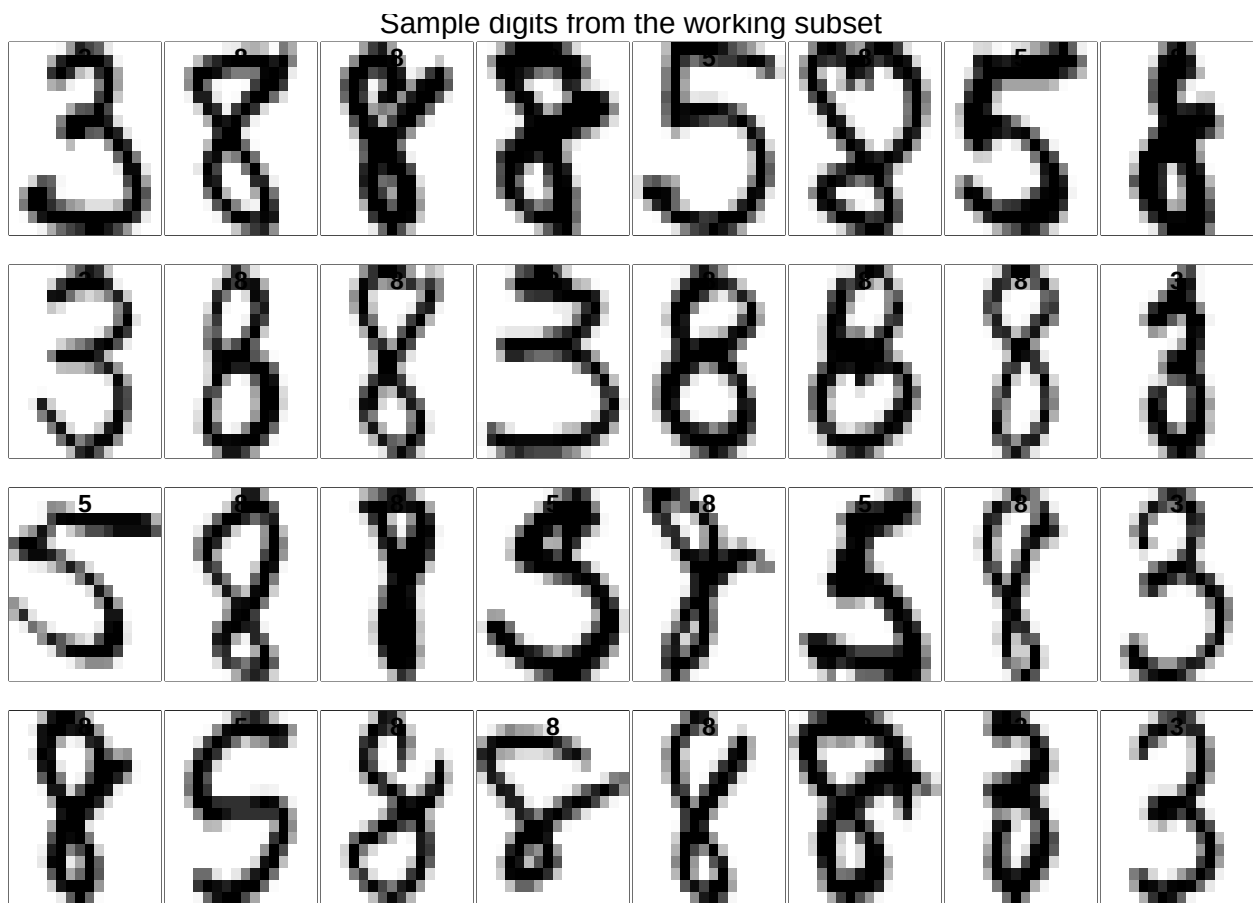
table(digit_lab)

## digit_lab

```

```
## 3 5 8
## 200 200 200
```

```
plot_digit_grid(dat_lab, labels = digit_lab, n = 32, main = "Sample digits from the working subset")
```



Starter code for Problem 1: PCA

The code below gives one reasonable PCA starting point. You should decide whether centering and scaling are appropriate for your question.

```
# Try changing these options and compare the results.
```

```
pca_center <- TRUE
```

```
pca_scale <- FALSE
```

```
pca_fit <- prcomp(dat_lab, center = pca_center, scale. = pca_scale)
```

```
# PCA scores for each digit image.
```

```
pca_scores <- pca_fit$x
```

```
# PCA loading vectors. Each column can be displayed as an image.
```

```
pca_loadings <- pca_fit$rotation
```

```
# Variance explained.
```

```
pca_var <- pca_fit$sdev^2
```

```
pca_prop_var <- pca_var / sum(pca_var)
```

```
pca_cum_var <- cumsum(pca_prop_var)
```

```

head(pca_prop_var)

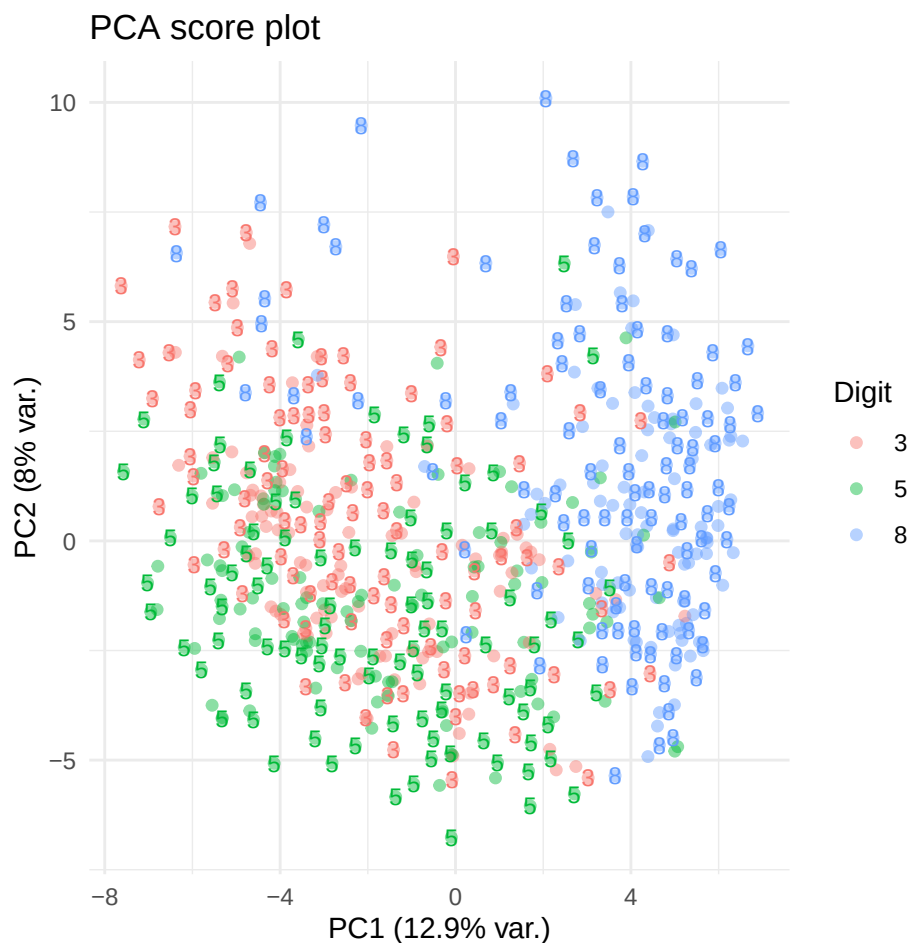
## [1] 0.12948374 0.08004089 0.07662554 0.05962934 0.05500896 0.03971890

which(pca_cum_var >= 0.95)[1]

## [1] 83

plot_embedding(
  pca_scores[, 1:2],
  labels = digit_lab,
  title = "PCA score plot",
  xlab = paste0("PC1 (", round(100 * pca_prop_var[1], 1), "% var.)"),
  ylab = paste0("PC2 (", round(100 * pca_prop_var[2], 1), "% var.)")
)

```



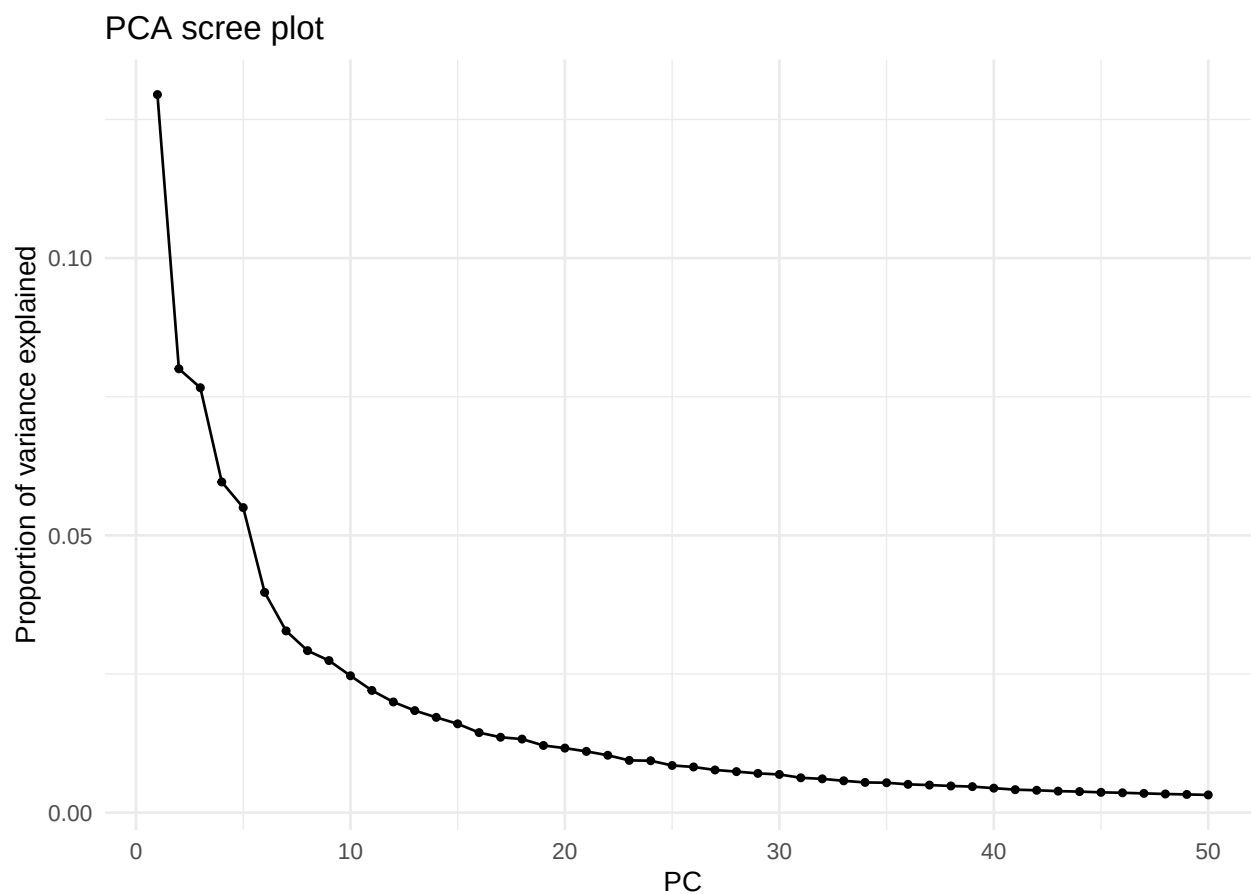
```

screedf <- data.frame(
  PC = seq_along(pca_prop_var),
  prop_var = pca_prop_var,
  cum_var = pca_cum_var
)

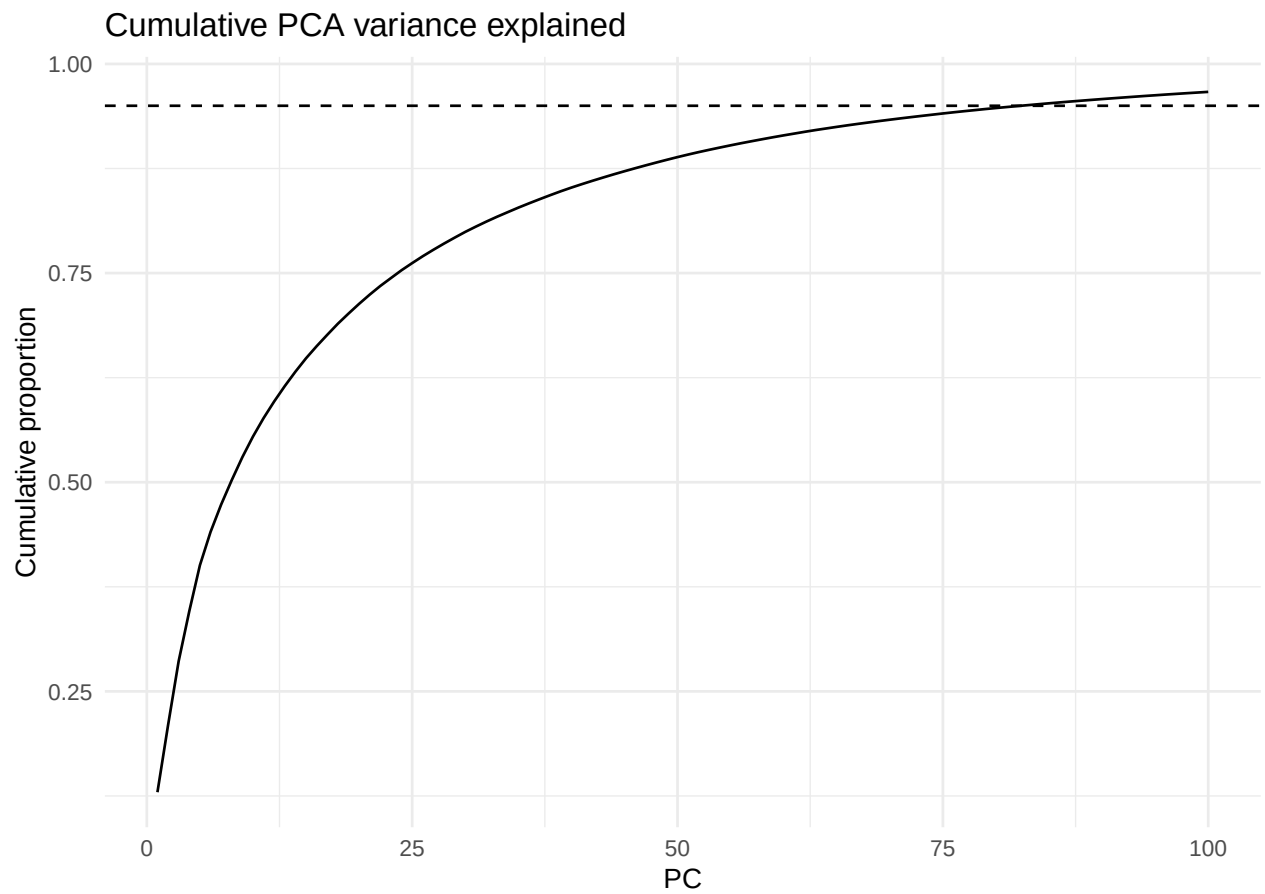
ggplot(screedf[1:min(50, nrow(screedf)), ], aes(x = PC, y = prop_var)) +
  geom_line() +
  geom_point(size = 1) +

```

```
labs(title = "PCA scree plot", y = "Proportion of variance explained") +
theme_minimal()
```

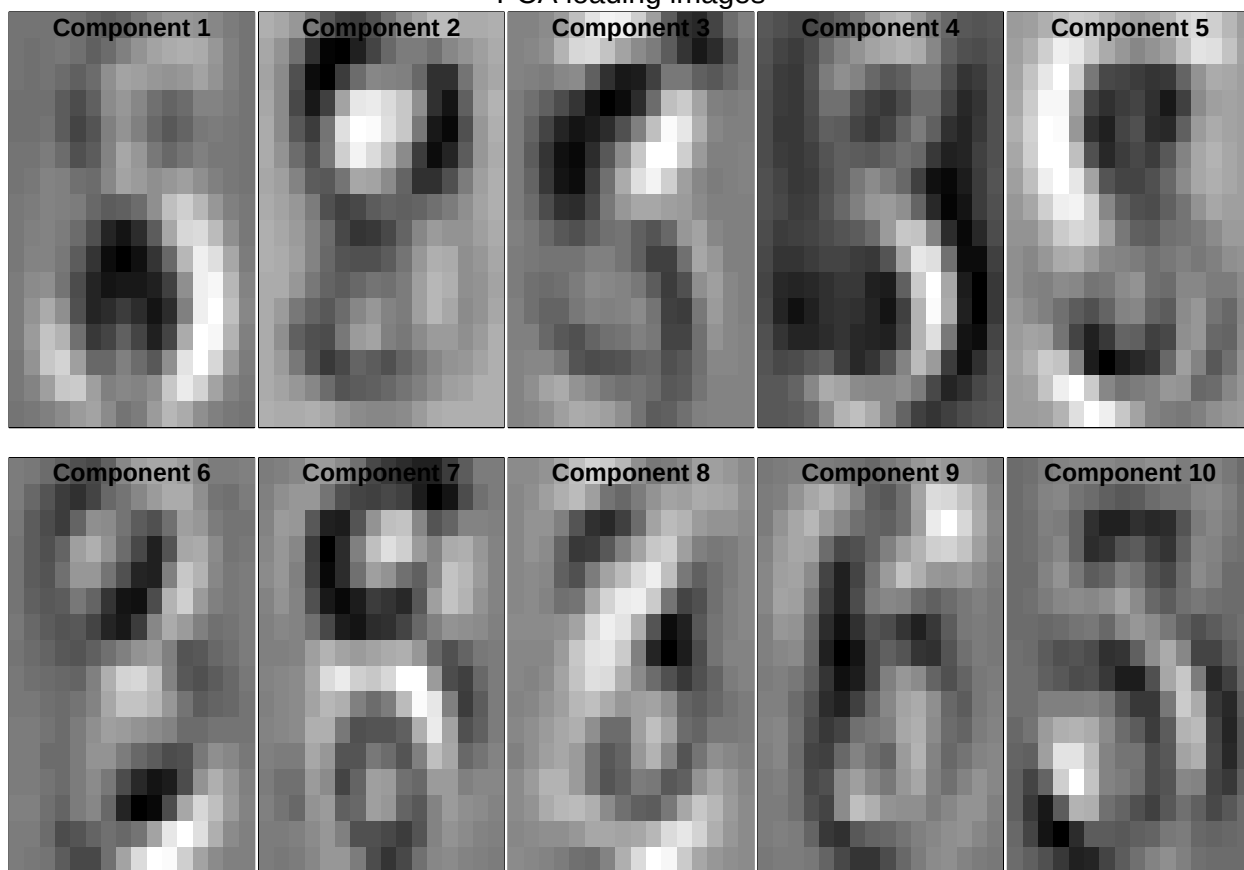


```
ggplot(screedf[1:min(100, nrow(screedf)), ], aes(x = PC, y = cum_var)) +
  geom_line() +
  geom_hline(yintercept = 0.95, linetype = "dashed") +
  labs(title = "Cumulative PCA variance explained", y = "Cumulative proportion") +
  theme_minimal()
```



```
# Inspect these loading images and describe the first five PCs in your own words.  
plot_basis_images(pca_loadings, n_show = 10, main = "PCA loading images")
```


PCA loading images



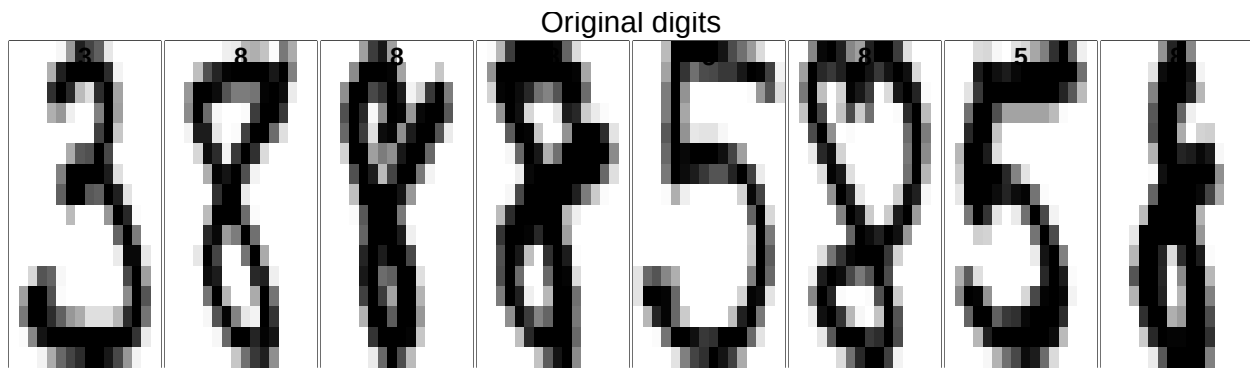
```
# Optional extension: change k_keep and compare the reconstructed images.
k_keep <- 10 # TODO: try 2, 5, 10, 25, 50, ...

X_center <- sweep(dat_lab, 2, pca_fit$center, FUN = "-")
if (is.logical(pca_fit$scale) && !pca_fit$scale) {
  X_scaled <- X_center
} else {
  X_scaled <- sweep(X_center, 2, pca_fit$scale, FUN = "/")
}

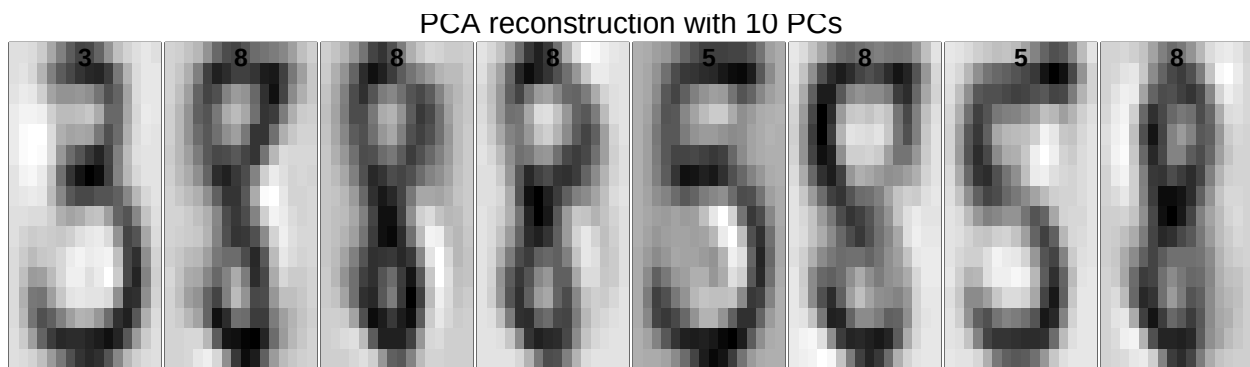
X_hat_scaled <- pca_scores[, 1:k_keep, drop = FALSE] %*% t(pca_loadings[, 1:k_keep, drop = FALSE])

if (is.logical(pca_fit$scale) && !pca_fit$scale) {
  X_hat <- sweep(X_hat_scaled, 2, pca_fit$center, FUN = "+")
} else {
  X_hat <- sweep(X_hat_scaled, 2, pca_fit$scale, FUN = "*")
  X_hat <- sweep(X_hat, 2, pca_fit$center, FUN = "+")
}

# Compare a few original and reconstructed digits.
example_ids <- seq_len(min(8, nrow(dat_lab)))
plot_digit_grid(dat_lab[example_ids, ], labels = digit_lab[example_ids], n = length(example_ids), main = )
```



```
plot_digit_grid(X_hat[example_ids, ], labels = digit_lab[example_ids], n = length(example_ids), main = )
```



Starter code for MDS

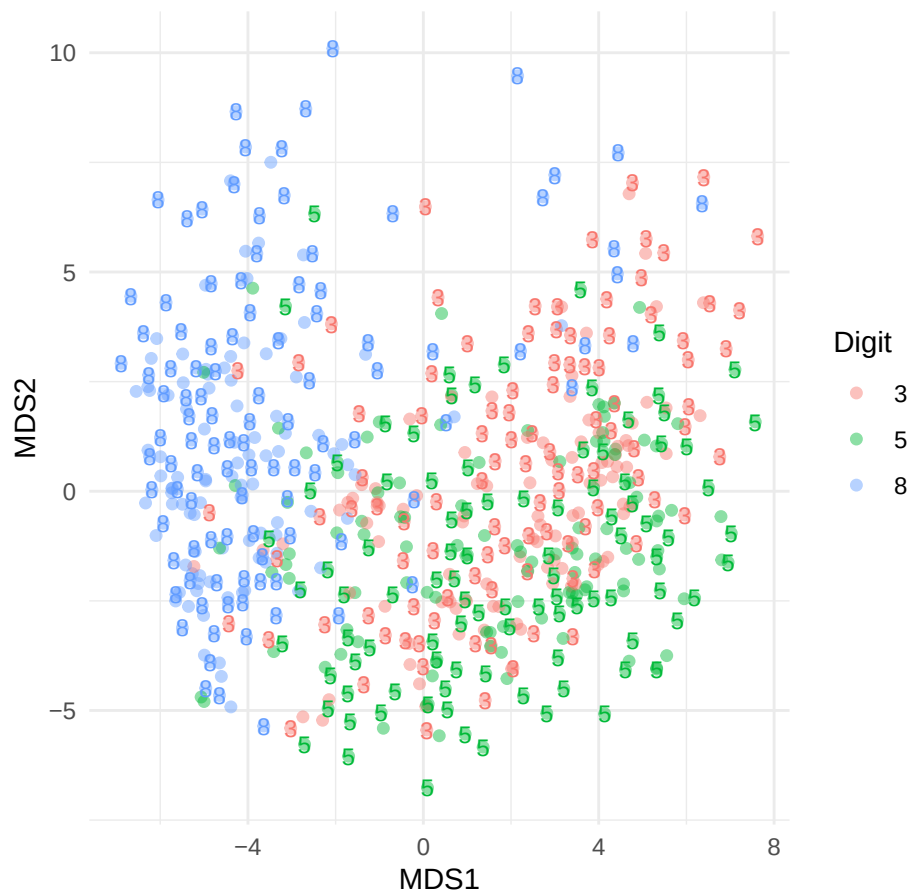
Classical MDS depends strongly on the distance matrix. The original lab used `method = "maximum"`, which is Chebyshev distance, not Manhattan distance. Use `method = "manhattan"` for Manhattan/L1 distance.

```
# TODO: try "euclidean", "manhattan", "maximum", and other distances supported by dist().
mds_distance <- "euclidean"
```

```
Dmat <- dist(dat_lab, method = mds_distance)
mds_fit <- cmdscale(Dmat, k = 2, eig = TRUE)
mds_embedding <- mds_fit$points

plot_embedding(
  mds_embedding,
  labels = digit_lab,
  title = paste("Classical MDS with", mds_distance, "distance"),
  xlab = "MDS1",
  ylab = "MDS2"
)
```

Classical MDS with euclidean distance



```
# Starter scaffold: uncomment and modify to compare distance metrics.
# distance_grid <- c("euclidean", "manhattan", "maximum")
# for (d in distance_grid) {
#   Dmat_d <- dist(dat_lab, method = d)
#   mds_d <- cmdscale(Dmat_d, k = 2)
#   print(plot_embedding(mds_d, labels = digit_lab, title = paste("MDS:", d)))
# }
```

Starter code for NMF

NMF requires nonnegative data. The code below shifts and rescales the digit matrix so all entries are nonnegative. You should explain why this is needed.

```
if (requireNamespace("NMF", quietly = TRUE)) {
  K_nmf <- 6 # TODO: try several values.

  dat_nmf <- dat_lab - min(dat_lab)
  if (max(dat_nmf) > 0) dat_nmf <- dat_nmf / max(dat_nmf)

  set.seed(1)
  nmf_fit <- NMF::nmf(dat_nmf, rank = K_nmf, nrune = 1)

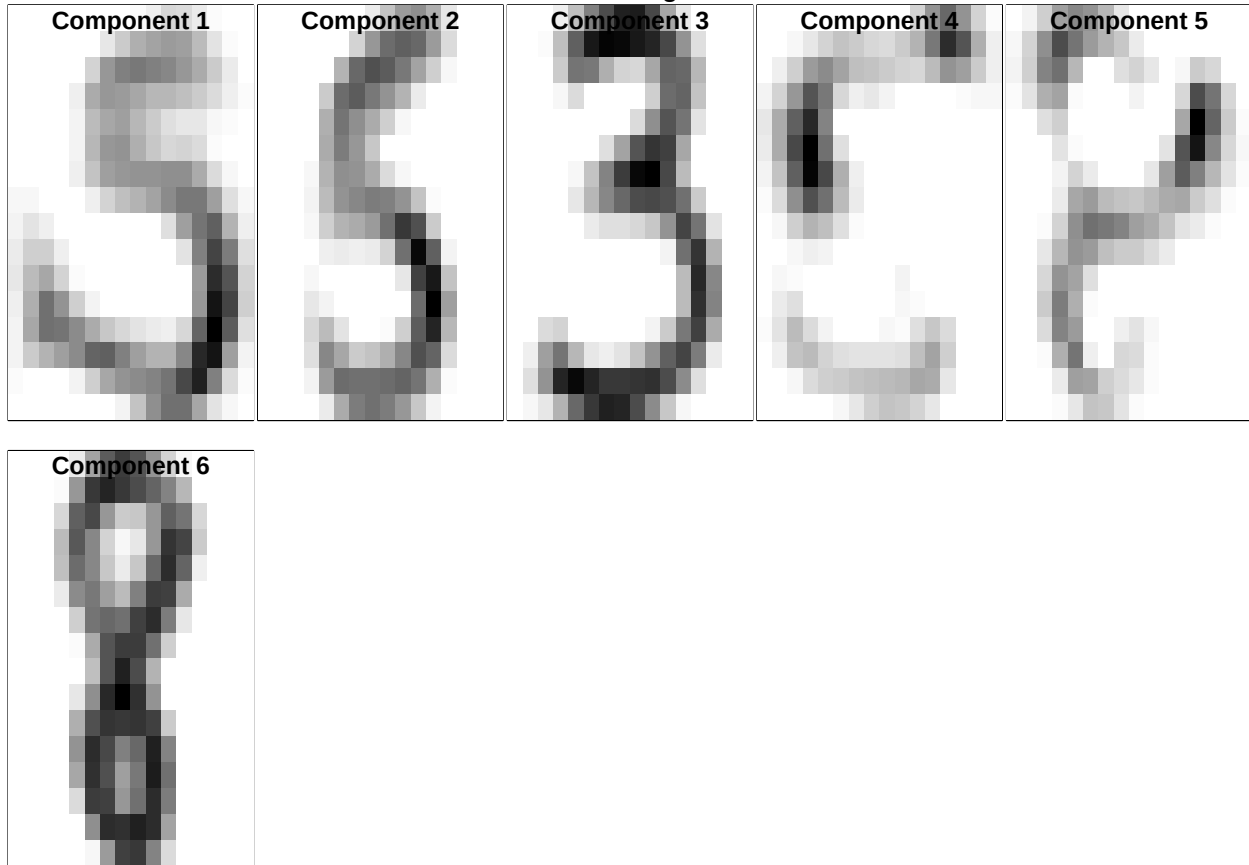
  W <- NMF::basis(nmf_fit) # sample weights
  H <- NMF::coef(nmf_fit) # basis images, one row per component
}
```

```

plot_basis_images(t(H), n_show = K_nmf, main = paste("NMF basis images, K =", K_nmf))
} else {
  message("Install the NMF package to run this section.")
}

```

NMF basis images, K = 6



Starter code for UMAP

UMAP results depend on the distance metric and the number of neighbors. Use a fixed random seed when comparing settings.

```

if (requireNamespace("umap", quietly = TRUE)) {
  set.seed(1)

  umap_metric <- "euclidean" # TODO: try "manhattan" or other supported metrics.
  umap_neighbors <- 15      # TODO: try smaller and larger values.

  umap_config <- umap::umap.defaults
  umap_config$metric <- umap_metric
  umap_config$n_neighbors <- umap_neighbors

  umap_fit <- umap::umap(dat_lab, config = umap_config)

  plot_embedding(
    umap_fit$layout,

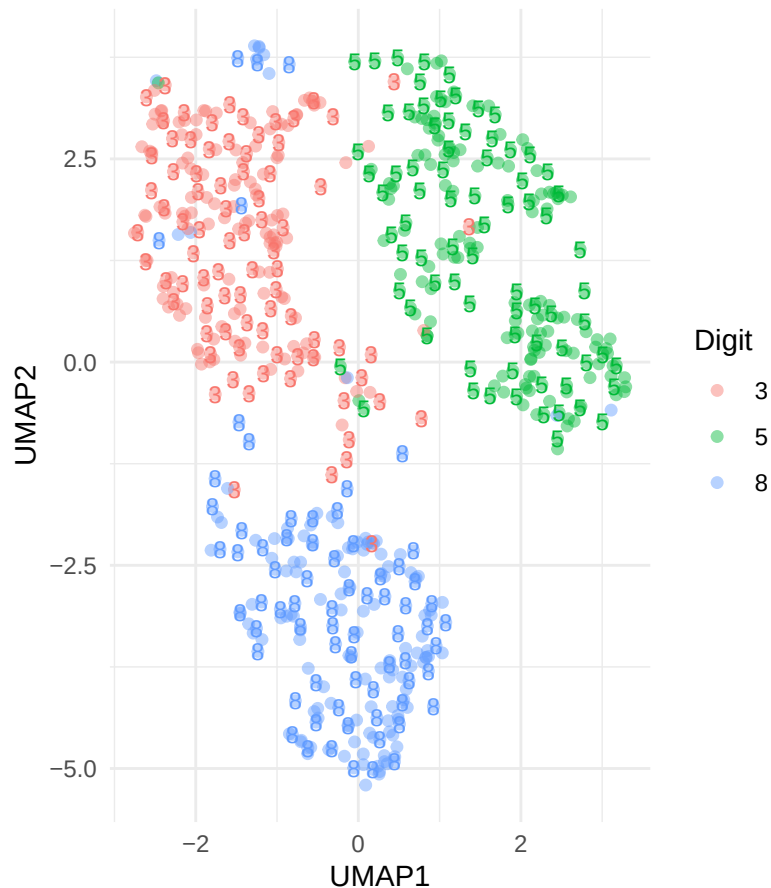
```

```

labels = digit_lab,
title = paste("UMAP:", umap_metric, "distance,", umap_neighbors, "neighbors"),
xlab = "UMAP1",
ylab = "UMAP2"
)
} else {
  message("Install the umap package to run this section.")
}

```

UMAP: euclidean distance, 15 neighbors



```

# Starter scaffold: uncomment and modify to compare numbers of neighbors.
# if (requireNamespace("umap", quietly = TRUE)) {
#   neighbor_grid <- c(5, 15, 50)
#   for (nn in neighbor_grid) {
#     cfg <- umap::umap.defaults
#     cfg$n_neighbors <- nn
#     cfg$metric <- "euclidean"
#     fit <- umap::umap(dat_lab, config = cfg)
#     print(plot_embedding(fit$layout, digit_lab, title = paste("UMAP neighbors =", nn)))
#   }
# }

```

Starter code for t-SNE

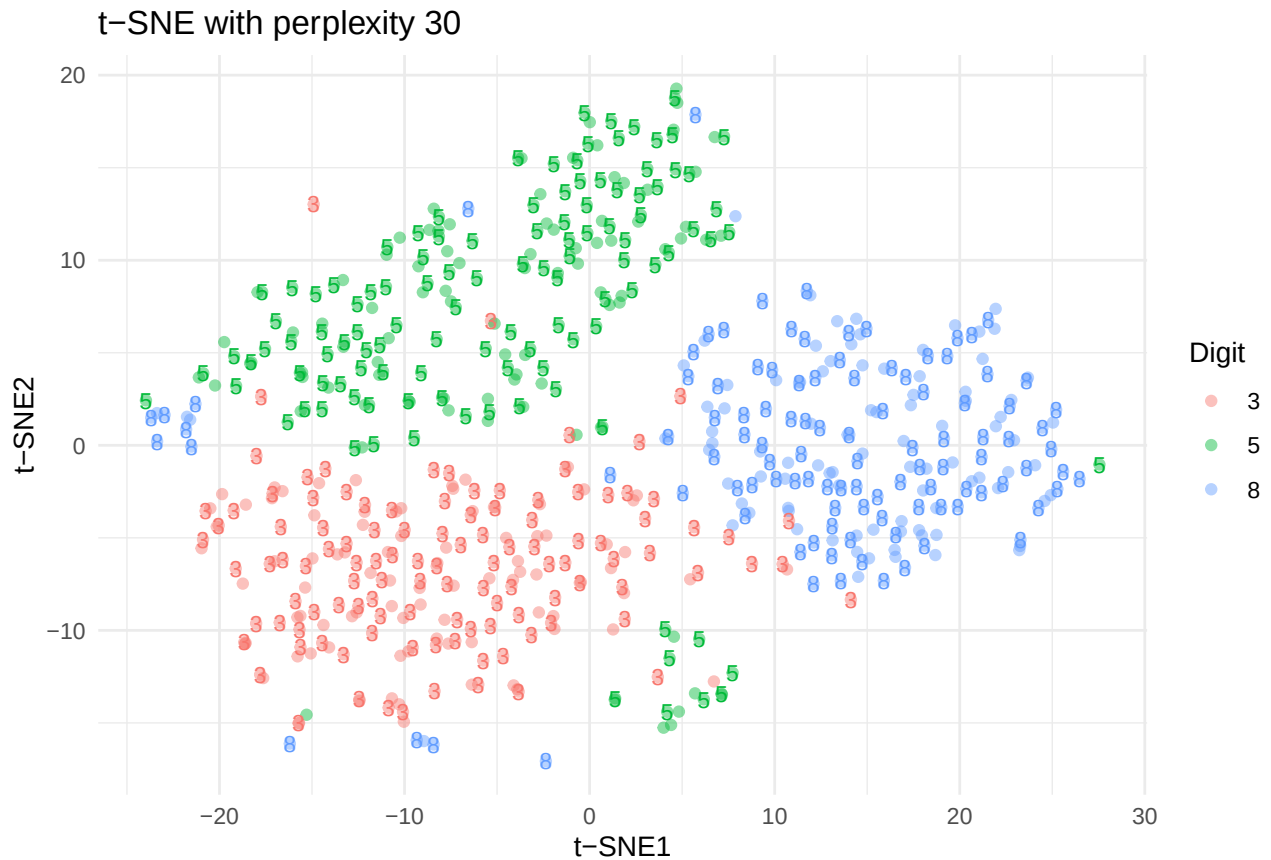
The t-SNE perplexity should be small relative to the sample size. If you use a very small subset of digits, large perplexities may fail.

```
if (requireNamespace("Rtsne", quietly = TRUE)) {
  set.seed(1)

  max_reasonable_perplexity <- floor((nrow(dat_lab) - 1) / 3)
  tsne_perplexity <- min(30, max_reasonable_perplexity)
  tsne_perplexity <- max(2, tsne_perplexity)

  tsne_fit <- Rtsne::Rtsne(
    as.matrix(dat_lab),
    dims = 2,
    perplexity = tsne_perplexity,
    check_duplicates = FALSE,
    verbose = FALSE
  )

  plot_embedding(
    tsne_fit$Y,
    labels = digit_lab,
    title = paste("t-SNE with perplexity", tsne_perplexity),
    xlab = "t-SNE1",
    ylab = "t-SNE2"
  )
} else {
  message("Install the Rtsne package to run this section.")
}
```



```
# Starter scaffold: uncomment and modify to compare perplexities.
# if (requireNamespace("Rtsne", quietly = TRUE)) {
#   perplexity_grid <- c(5, 10, 30)
#   perplexity_grid <- perplexity_grid[perplexity_grid < (nrow(dat_lab) - 1) / 3]
#   for (pp in perplexity_grid) {
#     set.seed(1)
#     fit <- Rtsne::Rtsne(as.matrix(dat_lab), perplexity = pp, check_duplicates = FALSE, verbose = FALSE)
#     print(plot_embedding(fit$Y, digit_lab, title = paste("t-SNE perplexity =", pp)))
#   }
# }
```

Starter code for Comparison table

Use this table as a template. Add rows, comments, and evidence from your plots.

```
comparison_table <- data.frame(
  method = c("PCA", "MDS", "ICA", "NMF", "UMAP", "t-SNE"),
  main_tuning_parameter = c(
    "number of PCs; center/scale choice",
    "distance metric; number of dimensions",
    "number of independent components K",
    "rank K; initialization",
    "distance metric; number of neighbors; min_dist",
    "perplexity; initialization; iterations"
  ),
  separation_quality = NA_character_,
```

```

interpretability = NA_character_,
notes = NA_character_
)

```

```

comparison_table

```

```

##      method                                main_tuning_parameter separation_quality
## 1      PCA                number of PCs; center/scale choice          <NA>
## 2      MDS            distance metric; number of dimensions          <NA>
## 3      ICA            number of independent components K          <NA>
## 4      NMF                rank K; initialization          <NA>
## 5      UMAP distance metric; number of neighbors; min_dist          <NA>
## 6      t-SNE      perplexity; initialization; iterations          <NA>
##      interpretability notes
## 1      <NA> <NA>
## 2      <NA> <NA>
## 3      <NA> <NA>
## 4      <NA> <NA>
## 5      <NA> <NA>
## 6      <NA> <NA>

```